

Course Outline: Containers (Without Docker)

Title: Containers (Without Docker) **Skill:** Skill 34 — Construir aplicaciones en contenedores **Learnpack:** + Contenedores (Sin hablar de Docker) **File:** s34-w12d35-contenedores-sin-hablar-de-docker
Prerequisite: Managing Database Evolution with FastAPI **Target Audience:** Beginner developers who have built FastAPI apps and understand ORM and migrations, but have not yet deployed or containerized anything **Total Lessons:** 10 **Format:** Conceptual (0% hands-on)

Course Description

You've built a FastAPI app, connected it to a database, and run migrations. Everything works perfectly — on your machine. But what happens when a teammate pulls the repo? Or when you deploy to a server? Suddenly Python versions clash, libraries are missing, and the app refuses to start. This course introduces containers: the technology that solves the "works on my machine" problem permanently. Before touching Docker, students learn what containers are, how they work internally, how they compare to virtual machines, and why they have become the standard for deploying modern AI and backend applications.

Course Philosophy

This course emphasizes:

- Understanding the problem before learning the tool — containers exist to solve a specific pain, and that pain must be felt before the solution makes sense
 - Conceptual depth over syntax — Docker commands come later; here students build the mental model that makes Docker obvious when they encounter it
 - Concrete analogies over abstract definitions — containers, images, and layers are explained through mental models that stick
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Container Concept	3
02 - Containers in Practice	3
03 - Assessment	1
04 - Outro	1
Total	10

00.0 Your App Works Here. But Not There. 📖

Learning Objectives:

- Recognize the environment consistency problem that containers solve
- Understand what this course will cover and how it connects to deployment

Content Outline:

- The scenario: a FastAPI app runs perfectly on the developer's machine, but crashes on a teammate's laptop or the production server
- Why this happens: different operating systems, different Python versions, different installed libraries, different environment variables
- The traditional "solutions" and why they fail: README instructions that go stale, `requirements.txt` that doesn't capture system-level dependencies, virtual environments that don't travel with the code
- The promise of containers: instead of configuring every machine to match your app, you package the app and its entire environment together
- Course preview: what containers are → how they work internally → how they compare to VMs → why they matter for AI and backend development

Transitions: Leads directly into Section 01, where we define what a container is and unpack the concept from the ground up.

Key Principles:

- The environment IS part of the application — shipping code without its environment is like shipping a recipe without the oven
- Containers don't eliminate the environment problem — they make it portable

Examples:

- Developer A uses Python 3.9 with a specific version of `pydantic`. Developer B has Python 3.11. The app behaves differently — or crashes — with no code changes.
 - A FastAPI app deployed to a Linux server fails because a C library it depends on is a different version than what was installed during development.
-

01.0 Containers 📖

Learning Objectives:

- Describe what a container is in plain terms
- Explain the core idea that containers package an app together with its environment
- Preview how this section breaks down the container concept

Content Outline:

- What a container is: a self-contained, isolated unit that bundles an application together with everything it needs to run — the runtime, libraries, dependencies, and configuration

- The key insight: a container is not just the code. It is the code + its environment, treated as one inseparable unit
- Containers are lightweight and run identically regardless of the host machine
- This section will cover: how containers work internally (images, layers, isolation) → how they compare to virtual machines
- Why this matters: once you understand what a container IS and how it works, using tools like Docker becomes natural and obvious

Transitions: Leads into 01.1, which goes inside the container to explain images, layers, and how a running container is actually created and isolated.

Key Principles:

- A container is both the application and its environment, packaged together
- Containers are designed to run identically everywhere — on any machine that supports the container runtime

Examples:

- A shipping container analogy: a physical shipping container holds goods and protects them from the outside environment. It can be loaded onto any ship, truck, or train and the contents remain the same. A software container works the same way — it holds your app and its environment, and runs identically on any machine with a container runtime.
- A Python FastAPI app packaged as a container includes Python itself, all pip packages, configuration files, and the application code — all in one portable unit.

01.1 How Containers Work

Learning Objectives:

- Explain the difference between a container image and a running container
- Describe how images are built from layers
- Understand how containers achieve isolation from the host system and from each other

Content Outline:

- **Container images:** A container image is a read-only snapshot of an app and its complete environment. Think of it as a template — a frozen, shareable blueprint that can be used to create running containers.
- **The image-to-container relationship:** An image is to a container what a class is to an object. You define the image once; you can run as many containers from it as you need. Each running container is an instance of the image.
- **Layers:** Images are built from stacked layers. Each layer represents one change — install Python, install packages, copy application files. Layers are cached and reused, making images efficient to build and share. Only the layers that changed need to be rebuilt or downloaded.
- **Container isolation:** When a container runs, it is isolated from the host operating system and from other containers. It has its own file system, its own network interface, and its own process

space — even though it shares the host OS kernel. The app inside the container cannot see or affect processes outside it.

- **The runtime:** A container runtime is the software that takes an image and runs it as an isolated container. The runtime is what makes containers portable — any machine with a compatible runtime can run any container image.

Anti-pattern — Treating a container like a virtual machine:

- Tempting because containers look like VMs from the outside — isolated environments running software
- What goes wrong: students try to run many processes inside one container (web server + database + scheduler), which defeats the purpose of isolation and makes containers harder to manage, scale, and debug
- The correct approach: one process per container. A FastAPI app in one container, the database in another. They communicate over a network.

Transitions: Now that we understand what containers are internally, the next lesson compares them to virtual machines — the technology containers are most often confused with.

Key Principles:

- An image is a blueprint; a container is a running instance
- Layers make images efficient — only changed layers need to be rebuilt
- Containers are isolated but not fully independent — they share the host OS kernel

Examples:

- An image for a FastAPI app might have three layers: (1) a base Python layer, (2) a layer installing the pip dependencies, (3) a layer copying the application code. If only the app code changes, only the third layer needs to be rebuilt.
- Running three containers from the same image gives you three fully isolated FastAPI instances, each with its own file system and network, all sharing the same host OS.

01.2 VMs vs Containers

Learning Objectives:

- Describe the architecture of a virtual machine and how it differs from a container
- Identify the key trade-offs between VMs and containers in terms of size, startup time, and isolation
- Know when VMs are the right choice and when containers are

Content Outline:

- **Virtual machine architecture:** A VM runs a full operating system inside a hypervisor. The hypervisor emulates hardware, and the guest OS sits on top of it. Everything the app needs — OS, kernel, drivers, libraries — is included in the VM.
- **Container architecture:** A container shares the host OS kernel. The container runtime (not a hypervisor) isolates processes using kernel features. No second OS is installed — only the app

and its user-space dependencies are packaged.

- **The practical differences:**

- **Size:** A VM image is typically gigabytes (includes a full OS). A container image is typically megabytes (just the app layer on top of a minimal base).
- **Startup time:** VMs take minutes to boot (a full OS must start). Containers start in seconds or milliseconds.
- **Isolation:** VMs offer stronger isolation — a compromised VM cannot affect the host kernel. Containers share the kernel, so a kernel-level vulnerability affects all containers on the host.
- **Resource usage:** VMs reserve memory and CPU for an entire OS. Containers share host resources dynamically and use only what the app needs.

- **When to use each:**

- VMs: when you need full OS isolation, different operating systems on the same host, or strong security boundaries (cloud providers running VMs for different customers)
- Containers: when you need fast, lightweight, consistent packaging of application workloads — the dominant choice for modern web apps, APIs, and AI services

Transitions: Now that we know what containers are and how they work, Section 02 turns to the practical question: why are containers so important for AI and backend development specifically?

Key Principles:

- Containers and VMs are not the same thing — containers share the host kernel, VMs do not
- Containers win on speed and efficiency; VMs win on isolation depth
- Most modern application deployment uses containers, often running inside VMs (the two technologies complement each other)

Examples:

- Cloud providers (AWS, Google Cloud) run your containers inside VMs — you get container convenience on top of VM-level isolation from other customers.
- A FastAPI app container starts in under a second. The equivalent VM running the same app might take 60 seconds to boot the OS before the app can even start.

02.0 Containers in Modern Development

Learning Objectives:

- Recognize why containers have become the standard for deploying backend and AI applications
- Preview how the "works on my machine" problem and the specific benefits of containers connect to real development workflows

Content Outline:

- The shift in how software is deployed: from configuring servers manually to shipping containers that carry their own environment
- Why backend and AI projects are especially vulnerable to environment inconsistency: Python version differences, native library dependencies (CUDA, C extensions), complex dependency trees

- The four ways containers solve these problems in practice — previewed here, detailed in the next two lessons:
 1. Eliminating the "works on my machine" problem
 2. Reproducibility: the same behavior everywhere, every time
 3. Simplified deployment: ship the container, not a configuration guide
 4. Dependency isolation and team consistency
- This section covers the "why it matters" — the next two lessons go deep on each

Transitions: Leads into 02.1, which examines the "works on my machine" problem in detail and shows exactly how containers solve it at the environment level.

Key Principles:

- The environment problem is not a personal failing — it's a structural problem with how software is traditionally deployed
- Containers don't just make deployment easier — they make it deterministic

Examples:

- An AI team trains a model in Python 3.10 with PyTorch 2.0. A colleague tries to reproduce the results on Python 3.11 with PyTorch 2.1 and gets different numbers. A container would have prevented this entirely.
 - A backend API works perfectly in development but crashes in production because the production server has a different version of a C library. With containers, the library is bundled — the production server doesn't matter.
-

02.1 The "Works on My Machine" Problem

Learning Objectives:

- Identify the specific causes of environment inconsistency in software projects
- Explain how containers eliminate environment inconsistency at its root
- Recognize the limitations of traditional solutions like `requirements.txt` and virtual environments

Content Outline:

- **The problem in detail:** Modern applications depend on dozens of things beyond the application code itself — the language runtime version, third-party libraries, system libraries, environment variables, file paths, OS-specific behaviors. When any of these differ between machines, the app breaks in ways that are hard to diagnose.
- **Why traditional tools fall short:**
 - `requirements.txt` lists Python packages but not the Python version itself, not system-level libraries, not OS dependencies
 - Virtual environments isolate Python packages but not the Python version, and they don't travel with the code — they must be recreated on each machine
 - README documentation describing how to set up the environment goes stale and is never complete

- **How containers solve it:** The container image captures the complete runtime environment — the OS layer, the language runtime, every library, every configuration file — and freezes it. The same image runs identically on any machine with a container runtime, regardless of what else is installed on that machine.
- **The shift in responsibility:** Instead of "configure your machine to match the app," the philosophy becomes "the app carries its own environment." The machine running the container only needs to run the container runtime — nothing else.

Anti-pattern — Relying solely on `requirements.txt` for reproducibility:

- Tempting because it's simple and already part of the Python workflow
- What goes wrong: `requirements.txt` captures package names and versions, but not the Python version, system libraries, or environment variables. Two developers with different Python versions will get different behavior even with identical `requirements.txt` files.
- The correct approach: use containers to capture the full environment, not just the package list. `requirements.txt` still has a role — inside the container definition, it installs packages into the container's isolated environment.

Transitions: With the core problem and solution understood, the next lesson explores the broader benefits containers provide for reproducibility, deployment, and team collaboration.

Key Principles:

- The environment problem is not about missing documentation — it's structural. Containers fix it structurally.
- "Works on my machine" is a symptom of the environment not traveling with the code. Containers make the environment part of the artifact.

Examples:

- Developer A has Python 3.9 installed. Developer B has Python 3.11. A `requirements.txt` with `pydantic==1.10` installs correctly on both, but `pydantic` behaves differently between Python versions. The app passes tests on A's machine and fails on B's — for reasons neither can easily diagnose.
- A data scientist uses a specific version of NumPy compiled against a specific BLAS library. Another team member installs the same NumPy version but gets a different BLAS backend. Matrix operations produce slightly different results, causing model outputs to diverge.

02.2 Benefits for AI and Backend Development

Learning Objectives:

- Explain how containers enable reproducibility across development, staging, and production environments
- Describe how containers simplify the deployment process
- Understand how dependency isolation prevents conflicts in multi-project development
- Recognize how containers enforce consistency across development teams

Content Outline:

- **Reproducibility:** A container image is immutable — once built, it produces the same behavior everywhere, every time. The same image that passed tests in CI runs in production. There is no "we thought it was the same" — it IS the same. For AI projects, this means model training and inference environments are identical, eliminating subtle numerical differences caused by library mismatches.
- **Simplified deployment:** Traditionally, deploying a backend app meant configuring the server — installing the right language version, setting up a virtual environment, installing system dependencies, configuring environment variables. With containers, the server needs only to run the container. The deployment command is the same regardless of what the app requires. The environment comes with the app.
- **Dependency isolation:** Different projects on the same machine can require conflicting versions of the same library. Without containers, this creates conflicts that require careful management. With containers, each project runs in its own isolated environment. Two projects requiring different versions of the same library coexist without conflict — they never see each other.
- **Team consistency:** Every developer on a team runs the exact same container. There is no "it works on my machine but not yours" — if it works in the container on one machine, it works in the container on every machine. Onboarding a new developer no longer means walking them through environment setup — they pull the image and run it.

Key Principles:

- Reproducibility is not just a convenience — it is the foundation of trustworthy AI and backend systems. A result you cannot reproduce is a result you cannot trust.
- Containers shift deployment from "configure the server" to "run the image" — a fundamental change in how applications are released
- Isolation by default: containers don't just isolate your app from other apps, they isolate it from the host machine entirely

Examples:

- A machine learning team deploys a model inference service as a container. The QA team, the staging environment, and production all run the same image. A bug found in staging is guaranteed to exist in the same form in production — and a fix tested in staging is guaranteed to work in production.
- A developer works on three different FastAPI projects simultaneously. Project A requires SQLAlchemy 1.4. Project B requires SQLAlchemy 2.0. Without containers, managing these on one machine requires complex virtual environment switching. With containers, each project runs in isolation — no conflict, no switching.
- A company hires a new backend developer. Instead of "install Python, create a virtual environment, install these 47 packages, set these environment variables, install this system library..." the onboarding is: "install Docker, run this command." The developer is productive in minutes.

Transitions: This completes the content of the course. The assessment is next — a chance to confirm understanding of containers, how they work, and why they matter.

03.0 What You Know Now 🧐

Learning Objectives:

- Recall what a container is and how it differs from a virtual machine
- Explain how container images and layers work
- Identify the causes of environment inconsistency and how containers solve them
- Apply the reproducibility, deployment, isolation, and consistency benefits to real scenarios
- Evaluate when containers are the right tool

Question Format: Scenario-based

Questions:

1. A teammate pulls your FastAPI repo, runs `pip install -r requirements.txt`, and gets a `ModuleNotFoundError` for a system library your app depends on. You've never seen this error. What is the most likely cause, and what would a container solve here? a) The teammate has a corrupted pip installation. Reinstalling pip would fix it. b) The `requirements.txt` is incomplete. Adding the missing package would fix it. c) The environment differs between machines — `requirements.txt` doesn't capture system-level dependencies. A container would bundle the system library along with the app, making this error impossible. d) The teammate is using a different text editor. The file encoding is the issue.
2. What is the relationship between a container image and a running container? a) They are the same thing — "image" and "container" are interchangeable terms. b) An image is a running instance; a container is a stopped image. c) An image is a read-only blueprint; a running container is an instance created from that image — like a class and an object. d) An image is stored in memory; a container is stored on disk.
3. A container image for a FastAPI app has three layers: a base Python layer, a layer installing pip packages, and a layer copying the app code. You update only the app code and rebuild the image. Which layers need to be rebuilt? a) All three layers — every rebuild starts from scratch. b) Only the pip packages layer, because dependencies may have changed. c) Only the app code layer — the Python and packages layers are cached and unchanged. d) None — once an image is built, it cannot be modified.
4. Your company runs AI model inference services. The model behaves correctly in the development environment but produces slightly different outputs in production. No code changes were made between environments. What is the most likely cause, and how would containers prevent it? a) The model is non-deterministic — this is expected. Containers wouldn't help. b) The production server has a different version of a numerical library (NumPy, BLAS). A container would ship the exact library version, making dev and production identical. c) The GPU in production is different. Containers cannot solve hardware differences. d) The model needs to be retrained on the production data. This is a data problem, not an environment problem.
5. A developer argues: "We don't need containers — we already use virtual environments and `requirements.txt`. That's good enough." What is the strongest counterargument? a) Virtual environments are slower than containers. b) `requirements.txt` and virtual environments are not supported on Linux servers. c) Virtual environments isolate Python packages but not the

Python version itself, not system libraries, and not OS-level dependencies. They don't travel with the code. Containers capture the complete environment, not just the package list. d) Virtual environments can only handle one project at a time, while containers handle multiple.

6. Your team is working on two FastAPI projects. Project A requires `sqlalchemy==1.4`. Project B requires `sqlalchemy==2.0`. Without containers, what problem do you face, and how do containers solve it? a) No problem — pip handles version conflicts automatically. b) The two projects conflict at the package level, requiring careful virtual environment management. Containers give each project its own isolated environment — both versions coexist without conflict. c) You must choose one version and update both projects to use it. d) Only one project can run at a time on the same machine.
7. Which of the following best describes how a container differs from a virtual machine? a) A container runs a full guest operating system; a VM does not. b) A VM is faster to start than a container. c) A container shares the host OS kernel and contains only the app and its user-space dependencies; a VM includes a full OS and runs on a hypervisor. d) VMs are used for development; containers are used for production only.
8. A new developer joins your team. The app is containerized. What does their onboarding look like compared to a non-containerized app? a) It's more complex — they need to understand containers before they can run anything. b) It's identical — containers don't affect the onboarding experience. c) It's simpler — instead of following a multi-step environment setup guide, they run one command to pull and start the container. The environment comes pre-configured. d) They still need to install all dependencies manually, but the container validates them.

Evaluation Criteria:

- 8/8: Complete mastery — ready to use containers confidently and explain them to others
- 6-7/8: Strong understanding with minor gaps — review the scenarios missed
- 4-5/8: Partial understanding — revisit Sections 01 and 02
- 0-3/8: Foundational review needed — start from the beginning

Score Interpretation: Containers are the foundation of modern deployment. Understanding them conceptually before picking up Docker tools will make everything that follows — Dockerfiles, images, Compose — immediately intuitive.

04.0 Outro

Content Outline:

- Course recap: what students accomplished
 - Understood why environment inconsistency exists and why it's a structural problem — not a setup failure
 - Learned what containers are: self-contained units that package an app together with its complete environment
 - Explored how containers work internally: images, layers, the runtime, and isolation

- Compared containers to virtual machines: the key trade-offs in isolation, size, and startup time
- Connected containers to real development problems: reproducibility, simplified deployment, dependency isolation, team consistency
- Connection to bigger picture: containers are not just a deployment tool — they are the standard packaging format for modern software. Every major cloud provider, CI/CD system, and deployment platform is built around them. Understanding containers is understanding the infrastructure that runs the modern web.
- Next steps: the next learnpack introduces Docker — the tool that makes building, running, and sharing container images accessible. With the conceptual foundation from this course, Docker commands will make immediate sense.
- Resources for further exploration:
 - The OCI (Open Container Initiative) specification — the standard that makes containers portable across runtimes
 - Docker documentation: [docker.com/get-started](https://docs.docker.com/get-started/) (when you're ready for the tool)
 - "Containers from Scratch" talk by Liz Rice (YouTube) — for the curious who want to see how Linux kernel features (namespaces, cgroups) power containers under the hood
- Encouragement: understanding containers puts you in the company of professional backend and platform engineers. Most developers use Docker for years without understanding what containers actually are. You now understand both — and that foundation will pay dividends when you hit real deployment problems.

Note: This is conclusion only — no theory, exercises, or assessment.
